CNN Classification on CIFAR-10 Dataset

1. Final Architecture and reason for choosing it

The final model uses a CNN because CIFAR - 10 images are small RBG images with special patterns. CNN is good because convolution filters stores relationships between pixels and features.

The architecture initiate with a data augmentation layer and then uses three blocks. The filters gradually increases from 32 to 64 and then to 128. Each block contains two Conv2D layers with "same" padding, batch normalization, Relu activation, max pooling and dropout. Then model uses Flatten, dense layer with 256 units, batch normalization, Relu, dropout and final dense layer with 10 softmax units.

This Architecture was selected because it balances model capacity and training cost. As we move layers have more filters because they need to learn abstract features. Batch normalization improves training stability. Max pooling reduces computation. Dropout plus L2 regularization reduces overfitting.

2. Preprocessing and Augmentation

The dataset is loaded using TensorFlowKeras. The code analyses the data by printing the image shapes, label shapes, pixel ranges, label ranges, and class count. It also save image grid so data can be visually check before training.

The pixel values ranges from 0 to 255. The code converts the image into float32 and divides by 255, which scales the input to the 0 to 1 range. This makes training more stable because the optimizer works with smaller and consistent values.

The labels are kept as plain integer class labels, so the model uses sparse_categorical_crossentropy. A 10 percent validation set is popped out of the training data using a split. The test set is not used during training or tuning and is saved for the final evaluation only.

Data augmentation is applied only to the training data using random horizontal flips, small rotations, and small zooms. This helps model see versions of the training images and reduces memorization. Augmentation is not applied to validate or test data because those sets must remain fixed for fair evaluation.

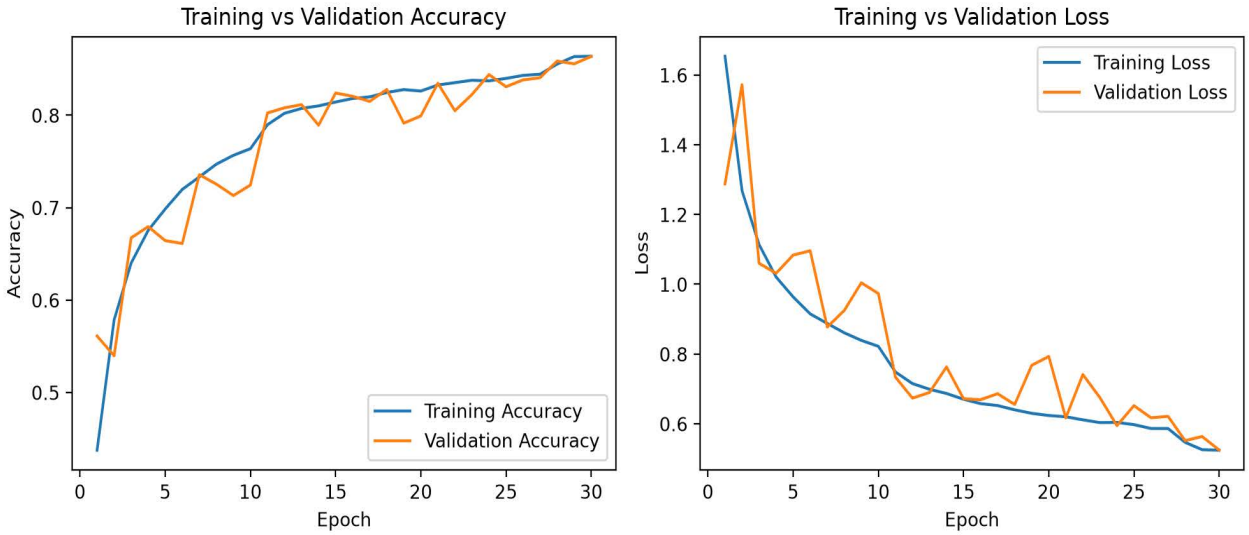
3. Training and validation curve Analysis

This model is compiled with Adam optimizer, learning rate is 0.001, sparse categorical cross-entropy and accuracy as the main metrics. It is trained with validation monitoring. The code save a training-curves plot containing training accuracy, validation accuracy, training loss and validation loss.

To have a healthy learning pattern, training accuracy and validation accuracy should increase and remain close. On the other hand training loss and validation loss should decrease together. Overfitting is shown when training accuracy increases but and validation accuracy stops improving. The code use dropout, L2 regularization, data augmentation, and learning rate reduction to reduce overfitting.

Underfitting is shown when training and validation accuracy remains low and flat. The 3 block CNN architecture was choosen to give model enough capability to avoid underfitting.

If the gap becomes large the model is overfitting and need adjustment.

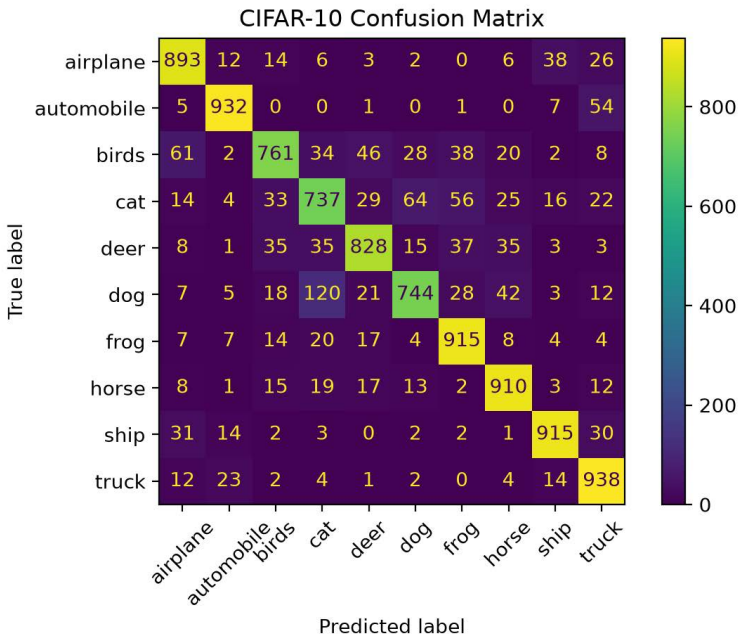


4. Final Test Accuracy

The test is evaluated after training and tuning are completed. This avoid data leaks and give a fair estimate.
Test Accuracy: 0.8575
Test Loss: 0.5605

5. Confusion Matrix

This pot gives more details than a single accuracy number because it shows which classes are predicted correctly and which are confused. When confusion matrix shows strong diagonal value for the class it means that the model learned distinctive shapes and backgrounds. If the classes are confused with each other then model is having difficulty in separating shapes. The classes having correct prediction vs incorrect prediction helps identify future improvements.



6. Conclusion

The project implements the CNN for CIFAR - 10 Dataset. The code loads and analyze different functions and features to train the, The final architecture was selected because it is strong enough to learn while still being capable enough for CPU training. The use of different feature and functions helps to improve generalization and overfitting